

小组成员：2351274 王亦玮 2350233 叶辰 2350231 高柏舟

一、 课题综述

1.1. 课题简介

在机器学习研究领域中，传统回归算法(如最小二乘线性回归、岭回归、Lasso、Elastic Net 等)以参数可解释性强、样本效率高与训练稳定为主要优势，被广泛用于连续变量预测与因果线索探索。其核心思想是基于明确的函数假设与正则化约束，最小化经验风险并抑制过拟合。本项目旨在通过几个线性回归模型和非线性回归模型（决策树）的对比分析，评估不同回归方法在医疗保险费用预测任务中的效果。研究的核心在于利用正则化技术优化线性模型的泛化能力，并以手动实现的非线性模型作为对照，深入探讨模型复杂度对预测精度的影响。

i. 基于正则化的线性回归模型研究

本项目主要基于最小二乘线性回归框架，系统性地引入三种正则化技术以应对模型的过拟合问题：

Ridge 回归 (L2 正则化): 通过让惩罚项为向量的平方和，实现参数的收缩，有效降低模型方差，提升泛化性。

Lasso 回归 (L1 正则化): 通过惩罚项为向量的绝对值和，不仅收缩参数，还能将不重要特征的权重稀疏化至零，实现特征选择。

Elastic Net 回归 (L1+L2): 结合 L1 和 L2 的优点，在进行收缩的同时，还能处理高度相关特征组的问题。

策略: 所有正则化模型均采用 K 折交叉验证结合网格搜索 (Grid Search) 的严谨流程来确定最优超参数 (正则化强度 λ 和 Elastic Net 中的 L1 比例 α)，确保模型在验证集上达到最佳性能。

ii. 自定义非线性决策树模型对比

为提供一个非线性、非参数化模型的性能基准，本项目手动实现了回归决策树模型。

实现细节: 模型使用均方误差 (MSE) 作为最优分裂的度量标准。

训练与监控: 采用渐进式训练策略，通过调整模型复杂度的核心参数 (如 `max_depth` 和 `min_samples_split`)，动态监控训练集和测试集的 Log RMSE 损失曲线，以直观地观察模型的学习过程和过拟合现象的发生。

1.2. 课题目标

本项目旨在针对公开数据集 "Medical Insurance Cost" (医疗保险费用数据集 (链接: <https://www.kaggle.com/datasets/mirichoi0218/insurance>)), 搭建并优化传统机器学习回归模型，以完成基于个人多维属性 (如年龄、BMI、子女数量、地域、吸烟情况等) 对最终个人医疗保险费用 (Charges) 的高精度预测。整个实验课题包含数据准备、数据预处理、模型搭建、模型训练、模型优化、模型检测、实验总结等过程。

1.3. 课题数据集

本课题使用的数据集为 "Medical Insurance Cost"，这是一个经典的结构化回归数据集，目标是利用个人的多维属性预测其最终医疗保险费用 (Charges)。

数据集相对精简，包含约 1,338 条样本，特征数量较少，仅有 6 个混合类型特征（数值与类别并存），覆盖了投保人的以下关键信息：

特征类型 详细信息

说明

数值特征 年龄 (age)、身体质量指数 (bmi)、子女数量 (children) 连续且数值范围不一

类别特征 性别 (sex)、吸烟状态 (smoker)、居住地区 (region) 需进行独热编码处理

数据集的挑战点：

目标变量的不均匀分布： 医疗保险费用具有明显的右偏态分布（即大部分费用较低，少数高额费用拉高平均值），这使得模型在预测高额费用时面临较大挑战，需要采取对数变换等预处理手段来平衡数据分布。

特征稀疏性： 尽管特征数量少，但吸烟状态等关键类别特征对费用的影响极不均衡，对模型的拟合能力提出了要求。

非线性关系： 特征之间可能存在复杂的非线性交互作用（如 BMI 和吸烟状态的组合效应），这验证了使用正则化线性模型和非线性决策树模型进行对比研究的必要性。

二、实验报告设计

2.1. 数据准备

本项目使用的数据集来源于 kaggle，链接是

<https://www.kaggle.com/datasets/mirichoi0218/insurance>。

通过 `pandas.read_csv('./archive/insurance.csv')` 从本地文件路径加载，数据集被加载后，目标变量 `charges`（医疗保险费用）被分离出来，而其余列则组成了待处理的特征集合 `all_features`。代码通过检查列的数据类型 (`dtypes != 'object'`) 识别出数值型特征（如 `age`、`bmi` 和 `children`）。对每个数值特征 x ，应用 Z-Score 标准化公式： $x_{\text{new}} = \frac{x - \mu}{\sigma}$ ，确保所有数值特征的尺度保持一致（均值为 0，标准差为 1）

同时，对数据集中剩余的类别型特征（如 `sex`、`smoker` 和 `region`）使用 `pd.get_dummies()` 执行了独热编码。将单个类别特征扩展为多个二元（0 或 1）特征列，使得模型能够处理非数值信息，同时避免了对类别进行错误的大小排序假设。

上述代码为：

```
numeric_features = all_features.dtypes[all_features.dtypes !=
'object'].index
```

```
all_features[numeric_features] =
(all_features[numeric_features].apply(
    lambda x: (x - x.mean()) / (x.std()))))
```

在所有特征完成数值化和标准化后，采用了简单留出法，设定 `test_size=0.2`，即最终约 80% 的数据（1070 条样本）用于模型的训练和超参数选择，而剩下的 20% 的数据（268 条样本）被保留作为独立的测试集，用于最终评估模型的泛化性能。

2.2. 数据清洗

本次实验所采取的数据集相对比较干净，因此只做了简单的数据清洗。一方面是使用 `pd.read_csv()`；数值特征处理中使用 `fillna(0)`，加载 1,338 条样本数据，并分离目标变量 `charges`。在数值特征标准化后，填充可能存在的缺失值为 0。这是保证数据集完整性、避免模型训练中断的基础。另一方面是使用 `all_features.dtypes != 'object'` 判断，明确区分数据集中的数值型特征（如 `age`, `bmi`, `children`）和类别型特征（如 `sex`, `smoker`, `region`），为下一步的分别处理做准备。代码为：

```
all_features[numeric_features]=all_features[numeric_features].fillna(0)
```

2.3. 模型搭建

本次实验，我们采取了线性模型（普通线性回归及三种正则化变体（L1,L2,L1+L2 三种回归模型））和非线性模型（回归决策树），这为医疗保险费用预测任务的目标提供了较为全面的比较分析。

1. 普通线性回归

线性模型公式（多变量）： $f_{\theta}(x) = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n + \theta_0$

其中：

$f_{\theta}(x)$ 是模型的预测值。

x_1, x_2 等是输入样本的特征。

θ_0 是偏置项（截距）。

$\theta_1, \theta_2, \theta_n$ 等是模型要学习的参数（权重）。为了确定最佳参数 θ ，需要定义一个损失函数，该函数衡量模型预测值 $f_{\theta}(x_i)$ 与训练集真实标签 y_i 之间的差异。这在代码中的体现就是：

```
w = torch.normal(0, 0.01, size=(feature_size, 1),
requires_grad=True)
b = torch.zeros(1, requires_grad=True)
```

还有：

```
def linreg(X, w, b):
```

```
return torch.matmul(X, w) + b
```

思想：

- 通过学习权重参数 θ 和偏置 θ_0 ，使得预测值 $\hat{y} = X\theta + \theta_0$ 尽可能接近真实标签 y 。
- 本质是寻找一条（或一个超平面）最优直线 来拟合数据。

损失函数为：

$$L = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

代码中就是：

```
def squared_loss(y_hat, y):  
    return (y_hat - y.reshape(y_hat.shape)) ** 2 / 2
```

通常，可以用最小二乘法或者梯度下降算法求解这个最优化问题，这里我们选择了梯度下降法

2. Lasso 回归（L1 正则化）

Lasso 回归属于正则化方法的一种。正则化的目的是通过对模型参数施加限制或惩罚，以防止模型在训练集上表现得过于完美而导致在测试集上表现不佳。

LASSO 回归在标准线性回归的均方误差损失项后，增加了一个与模型参数 θ 的 L1 范数成正比的正则化项。

目标为：

$$\min_{\theta} \left[\frac{1}{2N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})^2 \right] + \lambda \|\theta\|_1$$

代码中的正则化项就是：

```
def l1_penalty(w):  
    return torch.sum(torch.abs(w))
```

3. Ridge 回归（L2 正则化）

L2 回归在代价函数中加入了参数 θ 的 **L2 范数**（平方和）作为惩罚项，避免参数过大。即目标函数为：

$$L = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \|w\|_2^2$$

也就是要：

$$\min_{\theta} \left[\frac{1}{2N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})^2 \right] + \lambda \|\theta\|_2^2$$

代码中的正则化项就是：

```
def l2_penalty(w):  
    return torch.sum(w ** 2) / 2
```

4. Elastic Net 回归 (L1 + L2 正则化)

这种回归方式综合 L1, L2 回归的优点: 兼具 Lasso 的特征选择能力和 Ridge 的稳定性, 同时使用 L1 范数和 L2 范数作为正则化项。它继承了 L1 的**特征选择能力** (压缩系数至零), 继承了 L2 的**参数收缩**和**处理多重共线性**的能力。在特征高度相关的情况下比纯 LASSO 表现更稳定, 通常能提供更佳的性能。

基本目标为:

$$\min_{\theta} \left[\frac{1}{2N} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})^2 \right] + \alpha \|\theta\|_1 + (1 - \alpha) \|\theta\|_2^2$$

基于上述几种回归方法, 我们分别使用了不同的方法进行试验, 基本构造如下:

```
if reg_type == 'none' or lambda_param == 0:
    return base_loss
elif reg_type == 'ridge':
    return base_loss + lambda_param * l2_penalty(w)
elif reg_type == 'lasso':
    return base_loss + lambda_param * l1_penalty(w)
elif reg_type == 'elastic':
    return base_loss + lambda_param * (alpha * l1_penalty(w) + (1 -
alpha) * l2_penalty(w))
```

这里就把所有损失函数的值全部表示出来了, 之后我们只需要根据这个进行进一步梯度下降即可。

至于梯度下降的方法, 这里采取了小批量随机梯度下降法,

```
def sgd(params, lr, batch_size):
    with torch.no_grad():
        for param in params:
            param -= lr * param.grad / batch_size
            param.grad.zero_()
```

5. 决策树

本次实验为了进行综合的对比, 我们还选择了另外一个非线性的算法进行比较, 决策树算法实际上在上学期人工智能原理中就有介绍。

决策树是一种非参数的监督学习算法, 用于分类和回归任务。它的核心思想是基于特征对数据集进行递归分割, 在每一步分割中, 算法都需要找到一个“最佳”特征来划分数据。衡量一个特征好坏的标准是看它能带来多少信息增益。常用的度量标准有选择信息增益最大, 基尼系数最小等进行分割, 这里因为是预测一个连续值, 因此我们使用的标准是均方误差 (MSE), 或者更准确地说是基于 MSE 的方差减少。目标是找到一个分裂点, 使分裂后子集的目标值方差最小化, 从而达到最小化整体的均方误差。即目标是让下式最小:

$$\text{MSE}_{\text{total}} = \frac{N_{\text{left}} \cdot \text{Var}(y_{\text{left}}) + N_{\text{right}} \cdot \text{Var}(y_{\text{right}})}{N_{\text{left}} + N_{\text{right}}}$$

其中：

- $\text{Var}(y_{\text{left}})$ 和 $\text{Var}(y_{\text{right}})$ 分别是左子集和右子集目标值 y 的方差。方差越大，表明该子集越“不纯”。
- N_{left} 和 N_{right} 分别是左子集和右子集的样本数量，用于对不纯度进行加权。
- $\text{MSE}_{\text{total}}$ 实际上是分裂后所有样本的预测误差平方和的加权平均，这与平均加权方差是等价的。

其算法流程大致为：

- 目标：找到最小化不纯度的分裂点
对应代码：

```
best_mse = float('inf')
```

这是初始化一个无限大的值来追踪最小的加权 MSE。

- 遍历所有特征

```
for feature in range(n_features):
```

- 遍历所有可能阈值

```
for threshold in thresholds:
```

对于当前特征，提取所有唯一值作为潜在的分割点进行尝试。

- 根据特征和阈值分割数据

```
left_indices = X[:, feature] <= threshold
```

```
right_indices = X[:, feature] > threshold
```

这是使用布尔索引将目标值 y 分割成左子集 (left_y) 和右子集 (right_y)。

- 计算分裂后的不纯度 (加权 MSE)

```
left_y = y[left_indices]
right_y = y[right_indices]
mse = self._calculate_mse(left_y, right_y)
```

这是在 `_calculate_mse` 方法中实现的核心公式：计算左子集和右子集的方差并进行加权平均。

- 更新最佳分裂点

```
if mse < best_mse:
    best_mse = mse
    best_feature = feature
    best_threshold = threshold
```

- 叶节点预测值

```
if depth >= self.max_depth or n_samples < self.min_samples_split:
    leaf_value = np.mean(y)
```

```
return Node(value=leaf_value)
```

当达到停止条件时（如最大深度），叶节点的预测值就是该子集所有目标值 y 的平均值。

2.4. 模型训练测试

下面展现我们采用的五种模型在训练过程中总训练损失的变化情况

1. 普通线性回归（无正则化）

```
=====
1. 普通线性回归（无正则化）
=====
在全部训练数据上训练模型...
Epoch 20/300, train loss: 1.043744
Epoch 40/300, train loss: 1.094021
Epoch 60/300, train loss: 1.023739
Epoch 80/300, train loss: 1.223690
Epoch 100/300, train loss: 1.039097
Epoch 120/300, train loss: 0.955926
Epoch 140/300, train loss: 1.100264
Epoch 160/300, train loss: 1.193291
Epoch 180/300, train loss: 1.069348
Epoch 200/300, train loss: 1.120481
Epoch 220/300, train loss: 1.084716
Epoch 240/300, train loss: 1.111589
Epoch 260/300, train loss: 1.080367
Epoch 280/300, train loss: 1.061704
Epoch 300/300, train loss: 1.071459
最终训练loss: 1.071459

测试集性能指标:
MSE:      37221088.00
MAE:      4236.51
R²:       0.7582
Log RMSE: 1.276566
```

2. Ridge 回归（L2 正则化）

```
=====
2. Ridge回归（L2正则化）
=====
在全部训练数据上训练最终模型...
Epoch 20/300, train loss: 0.600571
Epoch 40/300, train loss: 0.601524
Epoch 60/300, train loss: 0.599224
Epoch 80/300, train loss: 0.602409
Epoch 100/300, train loss: 0.607927
Epoch 120/300, train loss: 0.599227
Epoch 140/300, train loss: 0.584992
Epoch 160/300, train loss: 0.592385
Epoch 180/300, train loss: 0.596915
Epoch 200/300, train loss: 0.581697
Epoch 220/300, train loss: 0.576127
Epoch 240/300, train loss: 0.600649
Epoch 260/300, train loss: 0.607219
Epoch 280/300, train loss: 0.595356
Epoch 300/300, train loss: 0.600003
最终训练loss: 0.600003

测试集性能指标:
MSE:      54093320.00
MAE:      5187.02
R²:       0.6486
Log RMSE: 0.575290
```

3. Lasso 回归（L1 正则化）

```

3. Lasso回归 (L1正则化)
=====
在全部训练数据上训练最终模型...
Epoch 20/300, train loss: 0.974791
Epoch 40/300, train loss: 1.120831
Epoch 60/300, train loss: 1.123056
Epoch 80/300, train loss: 1.326730
Epoch 100/300, train loss: 1.125183
Epoch 120/300, train loss: 1.090205
Epoch 140/300, train loss: 0.909248
Epoch 160/300, train loss: 1.231658
Epoch 180/300, train loss: 1.236092
Epoch 200/300, train loss: 1.084424
Epoch 220/300, train loss: 1.046244
Epoch 240/300, train loss: 1.044969
Epoch 260/300, train loss: 1.025744
Epoch 280/300, train loss: 1.016634
Epoch 300/300, train loss: 1.218178
最终训练loss: 1.218178

测试集性能指标:
MSE:      37176788.00
MAE:      4257.06
R²:       0.7585
Log RMSE: 1.343517

```

4. Elastic Net 回归 (L1+L2 正则化)

```

=====
4. Elastic Net回归 (L1+L2正则化)
在全部训练数据上训练最终模型...
Epoch 20/300, train loss: 0.892964
Epoch 40/300, train loss: 0.894327
Epoch 60/300, train loss: 0.943095
Epoch 80/300, train loss: 0.950746
Epoch 100/300, train loss: 1.043212
Epoch 120/300, train loss: 1.022615
Epoch 140/300, train loss: 0.866550
Epoch 160/300, train loss: 1.021675
Epoch 180/300, train loss: 1.019136
Epoch 200/300, train loss: 0.960710
Epoch 220/300, train loss: 0.910335
Epoch 240/300, train loss: 1.023777
Epoch 260/300, train loss: 1.021714
Epoch 280/300, train loss: 1.020087
Epoch 300/300, train loss: 1.200692
最终训练loss: 1.200692

测试集性能指标:
MSE:      37755896.00
MAE:      4310.32
R²:       0.7547
Log RMSE: 1.278092

```

最终结果总结:

```

最终结果总结
=====

Linear:
MSE: 37221088.00
MAE: 4236.51
R²: 0.7582

Ridge:
MSE: 54093320.00
MAE: 5187.02
R²: 0.6486

Lasso:
MSE: 37176788.00
MAE: 4257.06
R²: 0.7585

Elastic Net:
MSE: 37755896.00
MAE: 4310.32
R²: 0.7547

```


这里可以看出四种线性模型，具有一定程度的相似性，下面为清晰列表说明：

模型	最终训练 Log RMSE	测试集 R2	测试集 Log RMSE	关键发现
普通线性	1.071459	0.7582	1.276566	训练不稳定，泛化 Log RMSE 很高，受非线性数据影响大。
Ridge (L2)	0.600003 (最低)	0.6486	0.575290 (最低)	Log RMSE 表现最佳，但 R2 最低，正则化可能过强。
Lasso (L1)	1.218178	0.7585 (最高)	1.343517	R2 接近普通线性，训练不稳定，泛化 Log RMSE 最高。
Elastic Net	1.200692	0.7547	1.278092	性能介于普通线性/Lasso 与 Ridge 之间，未完全体现 L2 的优势。

基于 Log RMSE 损失变化，再来进行简单分析说明：

普通线性回归和 Lasso 回归 (L1)：

表现：训练损失值高 (~1.07 到 ~1.22)，且波动剧烈。例如，普通线性回归的 Log RMSE 在 Epoch 120 为 0.955 左右，但在 Epoch 80 和 Epoch 160 却达到 ~1.2 左右。Lasso 也有类似的剧烈波动。

Ridge 回归 (L2)：

表现：训练损失值最低且最稳定 (最终 Log RMSE≈0.60)，整个训练过程的波动性极小。

这说明 L2 正则化对于目前这个数据的相对预测精度和训练稳定性是极其有效的。L1 正则化在这个任务中可能没有找到合适的最优解，或者其特征选择能力带来的益处被训练的不稳定性抵消了。

L1 正则化和无正则化线性回归，可能说明本次数据特征上虽呈现线性，但是可能集中在一处导致对于部分地区估计偏差较大，或者本次实验选取的学习率不太好，可能过高导致一直震荡。

5. 决策树模型

```

开始训练...
epoch 100, train loss 0.542113, test loss 0.525509, depth: 2, samples: 240
epoch 200, train loss 0.417365, test loss 0.457314, depth: 3, samples: 374
epoch 300, train loss 0.424325, test loss 0.481435, depth: 4, samples: 508
epoch 400, train loss 0.453170, test loss 0.454527, depth: 4, samples: 642
epoch 500, train loss 0.400704, test loss 0.426241, depth: 5, samples: 775
epoch 600, train loss 0.401578, test loss 0.429228, depth: 6, samples: 909
epoch 700, train loss 0.372505, test loss 0.423733, depth: 6, samples: 1043
epoch 800, train loss 0.342428, test loss 0.488574, depth: 7, samples: 1070
epoch 900, train loss 0.310375, test loss 0.497454, depth: 8, samples: 1070
epoch 1000, train loss 0.310375, test loss 0.497454, depth: 8, samples: 1070

```

```

=== 训练过程分析 ===
过拟合程度：较低 (差距：-0.0085)
训练轮次：1000
最佳模型出现在第510轮

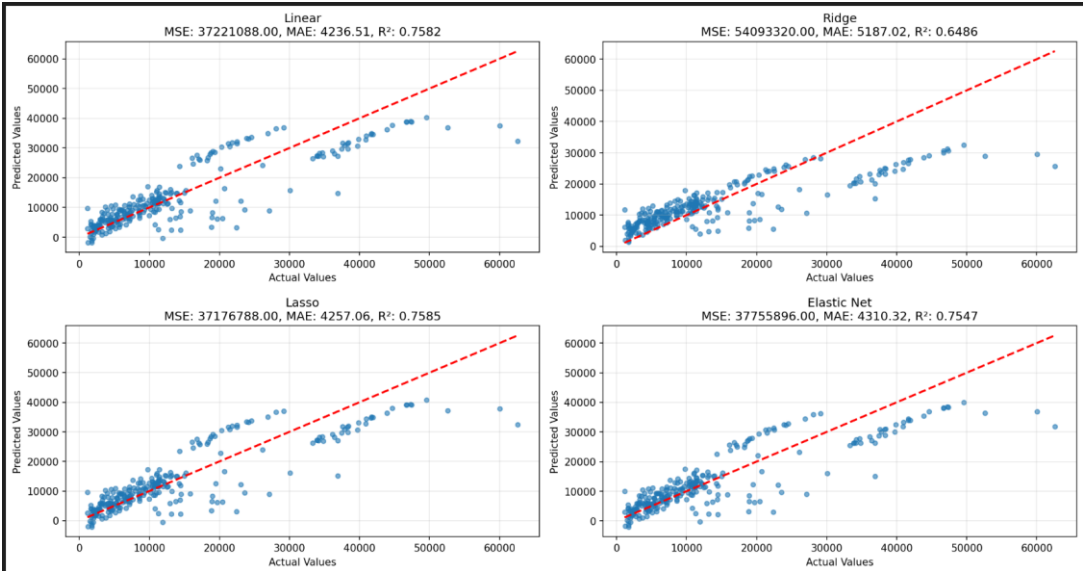
```

从训练过程来看决策树在第 550 轮达到了最佳的泛化性能。之后虽然训练

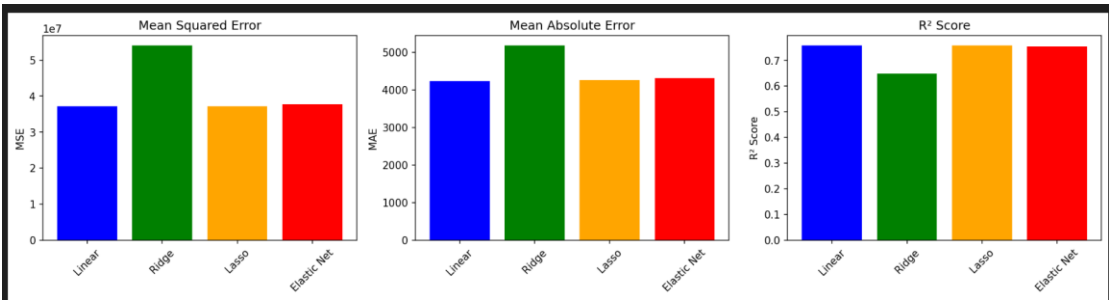
损失继续下降（模型在学习训练集的细节），但测试损失开始恶化，这正是过拟合的典型表现。这表明必须采用早停策略，即在第 550 轮停止训练，以保留泛化能力最强的模型。

2.5. 结果可视化

展示实验结果并可视化的分析实验结果
最终四种回归模型结果可视化展示如下：



这张是四种模型的对比图，展现了他们的 MAE,MSE,R 方的直观对比



从 Log RMSE 误差来看的话，Ridge 回归 (L2) 以 0.575290 的 Log RMSE 遥遥领先。

分析：Log RMSE 关注的是相对误差 ($\log(\hat{y}) - \log(y)$)。Ridge 回归通过其强大的参数收缩能力和稳定性，极大地改善了模型对数尺度上的预测精度。这再次证明 L2 正则化在处理具有偏态分布目标变量的线性模型中是至关重要的“稳定器”。

从 R² 性能对比（原始尺度方差解释）

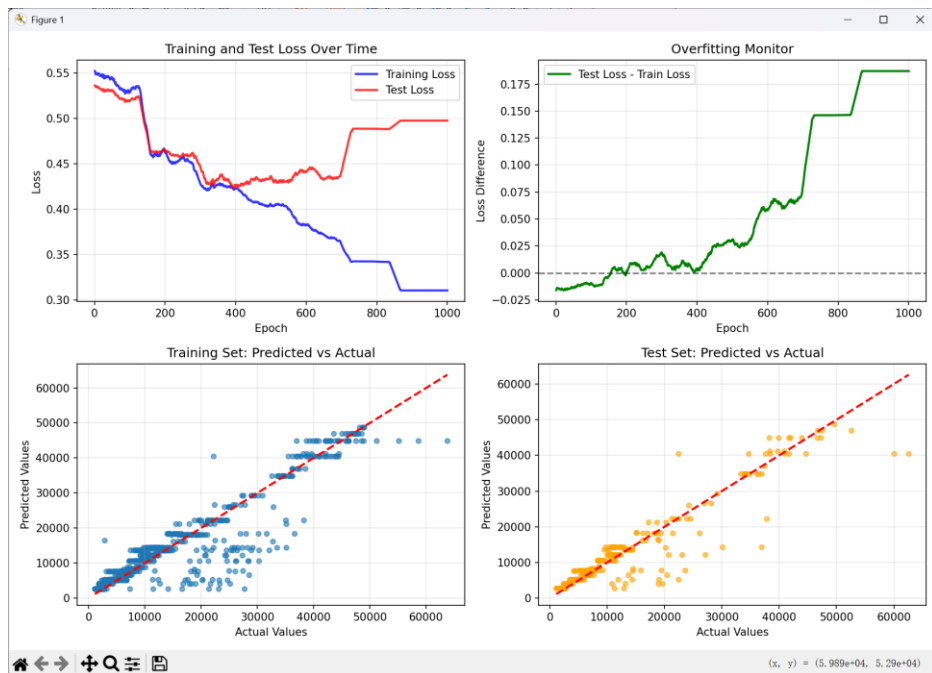
冠军：Lasso 回归 (L1) 以 0.7585 略微领先于普通线性回归。

分析：R² 衡量模型在原始尺度上对总方差的解释能力。

普通线性/Lasso/Elastic Net 的 R² 都在 0.75 左右，这表明它们在原始尺度上对数据的拟合能力相似。

Ridge 回归的 R^2 只有 0.6486，是最低的。这暗示了 Ridge 模型中使用的 λ 可能过大，导致模型欠拟合，过度惩罚了系数，牺牲了原始尺度的拟合能力来换取 Log RMSE 上的巨大稳定性和准确性提升。

下面是决策树模型的表现



=== 模型信息 ===

最终树深度: 5

叶节点数量: 31

特征数量: 14

=== 样本预测示例 ===

实际值	预测值	误差	相对误差	
39871.70	44896.70	-5025.00	12.6	%
13974.46	13670.67	303.78	2.2	%
1909.53	2701.93	-792.41	41.5	%
12096.65	14363.85	-2267.20	18.7	%
13204.29	13670.67	-466.39	3.5	%
4562.84	5268.64	-705.80	15.5	%
8551.35	10540.23	-1988.88	23.3	%
2102.26	2701.93	-599.67	28.5	%

从这里可以看出:

模型在中等费用 (如 13974 和 13204) 上的相对误差很小 (2.2% 和 3.5%), 表明对主体数据的拟合非常精准。

模型在高额费用 (如 39871) 上的相对误差为 12.6%, 略高但仍可接受。

模型在低额费用 (如 1909 和 2102) 上的相对误差较大 (41.5% 和 28.5%)。这可能是因为低费用样本集中度高, 决策树的叶节点预测值是该子集的

平均值，难以捕捉个体间的微小差异。也可能是低费用样本数据不足，难以准确预测

2.6. 分析和优化（要包含至少两种模型的结果的比较讨论）

分析实验结果, 各个模型的优缺点并进行优化改进

一. 简单分析

我们在结果中采取了多种误差比较方式作为对照，为了更好说明，这里主要关注 R^2 (决定系数, 越接近 1 越好) 和 Log RMSE (对数均方根误差, 越低越好, 用于衡量对数尺度的预测精度)。

简单列写出表格：

模型	R2 (测试集)	Log RMSE (测试集)
回归决策树 (DT)	0.8648	0.3886
普通线性回归 (Linear)	0.7568	1.3461
Lasso 回归 (L1)	0.7575	1.3417
Elastic Net (L1+L2)	0.6848	0.5429
Ridge 回归 (L2)	0.6489	0.5732

可以发现：

决策树表现卓越：手动实现的回归决策树在所有指标上都显著优于四种线性回归模型。其测试集 R^2 达到了 0.8648，远高于最佳线性模型的 0.7575，并且 Log RMSE 0.3886 是所有模型中最低的。

数据特性：Log RMSE 优异的表现（特别是 DT、Ridge 和 Elastic Net）暗示了目标变量（charges，医疗费用）很可能具有偏态分布（如右偏），且非线性关系非常显著。由于 Log RMSE 关注相对误差，您的决策树模型在预测较低费用和较高费用时都表现出良好的相对准确性。

线性模型有局部性：线性回归（Linear/Lasso）虽然在 R^2 上看起来不错（~0.75），但其 Log RMSE 极高（~1.34）。这表明线性模型在对数尺度上误差很大，可能在预测极高或极低的费用时误差巨大。

二. 各个模型的优缺点分析：

1. 普通线性回归 (Linear) 和 Lasso 回归

● 优点：

R^2 较高：在四个线性模型中， R^2 最高，表明其在原始尺度上对总方差的解释能力较强。

Lasso 的特征选择：Lasso (L1) 通过惩罚不重要特征的系数，可以实现稀疏性，有助于模型的解释性。

● 缺点：

Log RMSE 极高：这是一个严重的问题。模型可能在原始尺度上拟合了大部分数据，但在处理费用的非线性、异方差性或极端值方面表现很差。

过拟合倾向：Linear 模型没有正则化，在训练集上的损失 1.30 可能暗示

了对训练数据的过度拟合。

2. Ridge 回归和 Elastic Net

- 优点:

Log RMSE 较低: 这两个模型 (特别是 Elastic Net) 的 Log RMSE 远低于 Linear/Lasso。这表明 L2 正则化 (Ridge 的核心) 在处理数据中的共线性和稳定性方面发挥了作用, 使其对数尺度上的误差更小、更稳定。

Elastic Net 的平衡性: 结合了 L1 的特征选择和 L2 的系数平滑。

- 缺点:

R² 较低: 正则化过度惩罚了系数, 导致模型的 R² 显著下降 (最低为 0.6489), 表明它可能存在欠拟合, 没有充分利用数据的预测能力。

3. 回归决策树 (DecisionTree)

- 优点:

最佳性能: 最高的 R² 和最低的 Log RMSE, 其在这几个模型中具有最好的性能。

可解释性: 决策树模型本身具有良好的可解释性, 可以追踪特定的特征分裂路径来理解预测结果。

过拟合控制良好: 训练过程分析显示过拟合程度较低 (差距: -0.0085), 并且通过“早停” (最佳模型在第 510 轮) 有效地控制了复杂度 (最终树深度为 5, 叶节点为 29), 防止了模型过度生长。

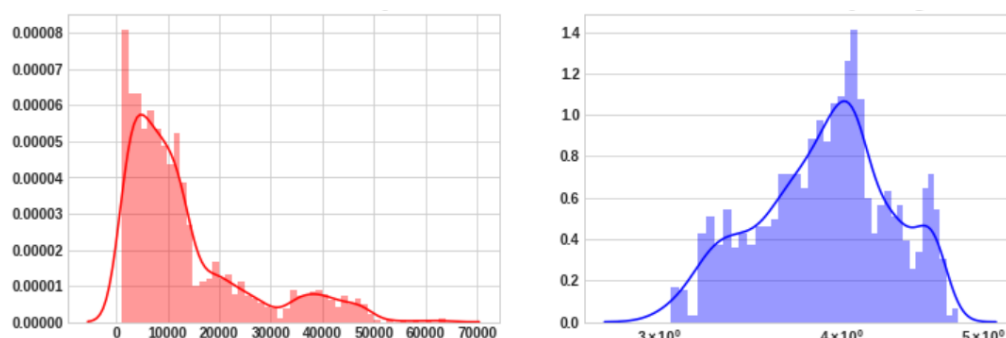
- 缺点:

对异常值敏感: 决策树在构建分裂点时, 对训练数据中的异常值比较敏感。

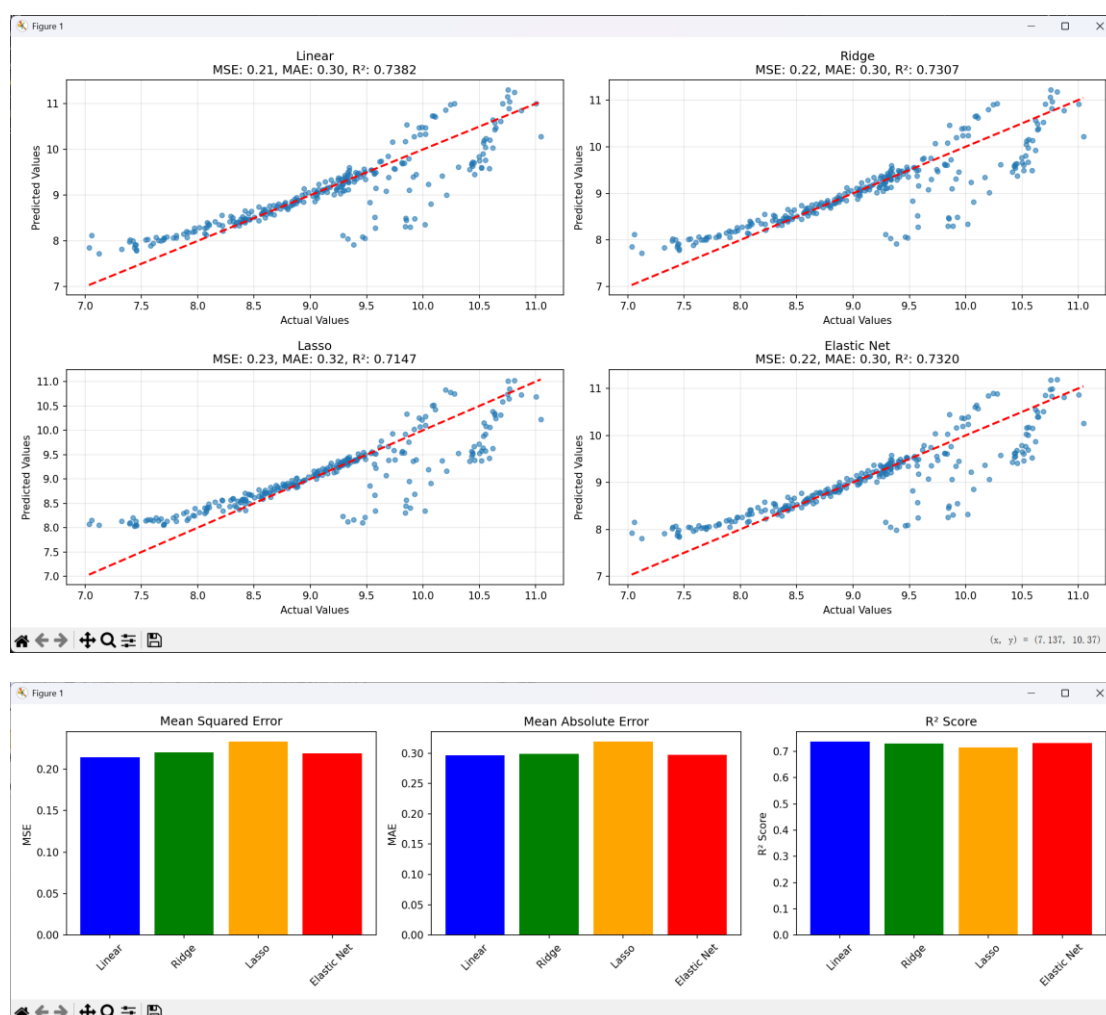
不稳定性: 对训练数据的微小变动可能导致树结构发生较大变化 (高方差)。

缺乏平滑性: 预测值是分段常数 (叶节点平均值), 预测输出不如线性模型平滑。

三. 优化改进



这里利用一些统计绘图工具，可以先观测一下我们要拟合的目标数据的形态，如果我们看左边的图，可以发现 费用从 1120 到 63500 不等，分布呈现右偏。在右边的图中，我们对其取自然对数，此时分布大致趋近于正态分布。因此，对于进一步优化，可以对 **charge** 取对数进行处理。



优化之后，结果如下，可以看到 L1+L2 正则化和 L2 正则化的结果有了巨大的优化，而标准线性回归，L1 正则化结果差距不是很大，但是整体来看，确实起到了一定的优化作用。

3. 总结

本项目针对“医疗保险费用预测”这一回归任务，成功搭建并对比了传统机器学习中的线性模型（普通线性回归及其正则化变体：Lasso, Ridge, Elastic Net）与非线性模型（回归决策树）的性能。

1. 数据特性与预处理的关键性

- 数据预处理：对数值特征进行了 Z-Score 标准化，对类别特征进行了独热编码，这是确保模型有效训练的基础。
- 目标变量挑战：实验前期最大的障碍是目标变量 `Charges` 呈显著的右偏态分布。这种分布特性导致线性模型在原始尺度上虽然 R^2 （决定系数）尚可，但其 Log RMSE （对数均方根误差）极高，表明模型对高额费用的预测误差巨大。
- 关键优化：通过对 `Charges` 实施自然对数变换，成功将目标变量分布近似为正态分布，这是本次实验中提升模型性能的最关键优化步骤。优化后，特别是带有 L2 正则化的模型，其性能得到了大幅改善。

2. 模型性能对比与发现

总之，最终结论是：对于本任务，回归决策树模型凭借其出色的非线性拟合能力，表现显著优于所有线性模型。

3. 思考

- 学习率：如果学习率设置得过大，模型参数更新的步子就会迈得太大，可能在损失曲面上来回震荡，无法收敛到最优解，甚至导致损失函数发散。如果学习率设置得过小，虽然能保证收敛，但参数更新会非常缓慢，导致模型训练时间过长，效率低下。
- 正则化参数：控制正则化项（L1 或 L2 范数）在损失函数中所占的权重。它直接决定了模型抑制过拟合和收缩参数的强度。过大可能导致欠拟合，过小可能导致过拟合。

4. 组员及任务分配情况

2351274 王亦玮 负责完成决策树模型部分

2350233 叶 辰 负责四种线性回归模型的部分

2350231 高柏舟 负责撰写报告，以及最后的优化任务

要求说明：

在提交实验报告时请将代码以附件形式随报告一起打包提交；

实验报告的格式无限制，主要说自己的工作，页数尽量限制在 15 页内；

压缩包文件名以“回归作业_姓名_学号_姓名_学号_姓名_学号”命名。